

[ANDROID SOFTWARE REVERSE ENGINEERING & DECOMPIlation](#)

The Anatomy of resources.arsc: Decoding Resource Types, Strings, and Values for Forensic Analysis

By Antigravity Research Published May 29, 2026 ⌚ 6 Min Read

GOOGLE ADSENSE NATIVE PLACEMENT - HORIZONTAL TOP-POST BANNER

Introduction to Android's Resource Core

The `resources.arsc` file stands as a critical component within every Android Application Package (APK). It acts as a highly optimized, compiled binary table that maps resource IDs to their corresponding values, types, and configurations (like language or screen density). For forensic analysts, reverse engineers, and security researchers, understanding its intricate structure is

paramount. It often holds hardcoded strings, hidden configurations, URLs, API keys, and other sensitive data that might not be immediately apparent from decompiled Java code or manifest files.

This article dives deep into the internal structure of `resources.arsc`, explaining its binary format, how resource types and strings are encoded, and how to effectively decode this information for forensic analysis, uncovering valuable intelligence.

The Binary Structure of `resources.arsc`

At its heart, `resources.arsc` is a collection of binary chunks, each defined by a common header structure: a `ResChunk_header`. This header typically specifies the chunk's type, the size of the chunk's header, and the total size of the chunk. The main chunks found within `resources.arsc` are:

1. Global String Pool: A central repository for all unique strings referenced by the resources in the file, preventing redundancy and optimizing storage.
2. Package Chunk: Represents a single Android package (APK). An `.arsc` file usually contains one Package Chunk. Each Package Chunk includes its own String Pool for resource names.
3. Type Spec Chunk: Defines a specific resource type (e.g., string, layout, drawable, color). It holds a list of flags indicating whether each resource entry of that type is public or private.
4. Type Config Chunk: Contains resource values for a specific configuration (e.g., `en-US` for English, `hdpi` for high-density screens).
5. Entry Chunk: The actual resource data, pointing either to a value directly or to an index within a String Pool.

This hierarchical structure allows Android to efficiently retrieve the correct resource based on the device's configuration and the requested resource ID.

The Global String Pool in Detail

The very first chunk in `resources.arsc` is often the global `ResStringPool_header`. This pool stores all literal string values used across different resource types. Accessing strings from this pool involves an index lookup. For example, an application's name, a button label, or a URL might all be stored here, referenced by their respective resource entries.

Decoding Resource Entries and Values

Every resource in Android is identified by a unique 32-bit integer, often seen in the format `0xPP TTTT EEEE`, where:

- `PP` : Package ID (e.g., `0x7f` for the application's package, `0x01` for Android system resources).
- `TTTT` : Type ID (e.g., `0x01` for ``string``, `0x02` for ``drawable``, `0x03` for ``layout``).
- `EEEE` : Entry ID within that type.

When Android needs a resource, it uses this ID to navigate through the `resources.arsc` file:

1. Locate the Package Chunk using `PP`.
2. Within the Package Chunk, find the Type Spec Chunk corresponding to `TTTT`.
3. Then, find the Type Config Chunk for the current device's configuration.

4. Finally, use `EEEE` to get the specific resource entry, which will either contain the value directly or an index to a string in one of the String Pools.

Practical Dissection with `aapt`

While manual binary parsing is complex, tools like the Android Asset Packaging Tool (`aapt` or `aapt2`) provide a high-level view of `resources.arsc` , which is often sufficient for initial forensic analysis.

First, extract the `resources.arsc` file from an APK. You can simply rename the APK to `.zip` and extract its contents, or use a tool like `unzip` :

```
unzip your_app.apk resources.arsc
```

Once extracted, you can use `aapt dump resources` to get a human-readable representation:

```
aapt dump resources your_app.apk
```

Or, if you only have the `resources.arsc` file:

```
aapt dump resources --values resources.arsc
```

The output will list all resources, their IDs, types, configurations, and values. Let's look for a hypothetical application name and a hidden URL:

```
Resource ID #0x7f010000 type #0x01: string, entries=1128value=(string) "My Awesome App"Resource ID #0x7f010001 type #0x01: string, entries=1128value=(string) "http://www.example.com/hidden-url"
```

In this simplified output, we can see the application name and potentially a command-and-control server URL, both stored as string resources. Forensic analysts can grep through this output for keywords, URLs, email addresses, or specific patterns often indicative of malicious behavior.

Beyond `aapt`: Automated Tools and Decompilers

For more in-depth or automated analysis, especially when dealing with obfuscated applications, tools like `apktool` are invaluable. `apktool` decompiles `resources.arsc` into human-readable XML files (`public.xml` and various `strings.xml` , `layouts.xml` , etc., within the `res` directory). This transformation makes it much easier to search, parse, and analyze resources programmatically.

```
apktool d your_app.apk -o decompiled_app
```

After decompilation, navigate to `decompiled_app/res/values/` to find `strings.xml` , `public.xml` , and other resource files. These XML files provide a clear mapping of resource IDs to their names and values, making it trivial to search for suspicious strings.

```
<!-- Example from decompiled strings.xml --><resources>    <string name="app_name">My Awesome App</string>
```

This human-readable format greatly simplifies the task of identifying embedded secrets or indicators of compromise.

Forensic Significance and Analysis Techniques

The `resources.arsc` file is a treasure trove for forensic analysis:

- **Hardcoded Credentials and API Keys:** Often, developers (or attackers) hardcode sensitive information directly into string resources. Searching for patterns like `API_KEY`, `password`, `username`, or specific key formats can reveal these.
- **Command and Control (C2) URLs:** Malware frequently stores C2 server addresses or communication endpoints as string resources, sometimes obfuscated.
- **Hidden Configurations:** Applications might have different behaviors based on values defined in resources, which can be critical for understanding an app's full capabilities or malicious intent.
- **Language-Specific Payloads:** Malware might deploy different payloads or messages based on the device's locale, all defined within localized resource configurations.
- **Package Names and Signature Information:** While not directly in `resources.arsc`, related information can be inferred or cross-referenced, helping to identify repackaged apps.

For advanced scenarios, especially when basic `aapt` or `apktool` analysis isn't enough (e.g., highly obfuscated or custom-packed `resources.arsc` files), analysts might resort to:

- Hex Editor Analysis: Manually inspecting the binary file for string patterns, albeit tedious.
- Custom Parsers: Writing scripts (e.g., in Python) to parse the `ResChunk_header` and navigate the binary structure, particularly useful for custom obfuscation schemes.

Conclusion

The `resources.arsc` file, though seemingly just a compiled resource table, is a fundamental component for gaining a deep understanding of any Android application. Its binary structure, while complex, systematically organizes all static assets. By leveraging tools like `aapt` and `apktool`, and understanding the underlying format, forensic analysts can effectively decode resource types, strings, and values. This decoding process is often critical for uncovering hidden data, identifying malicious indicators, and ultimately enhancing the overall security posture and investigative capabilities within the Android ecosystem.



Android Mobile Specs & Compare Directory

Are you researching mobile hardware properties, processor SoCs, GPU chipsets, or RAM configurations? Access our complete specs catalog to compare up to 5 devices side-by-side!

[Compare Devices Specs →](#)

Tags: [#Android Forensics](#) [#APK Analysis](#) [#resources.arisc](#) [#Reverse Engineering](#) [#Technical Guide](#)

Related Technical Guides

**Cracking Native Android
Obfuscation: Advanced JNI
Reverse Engineering
Techniques**

May 30, 2026

**Emulating & Reversing MIPS
Android Apps on x86
Systems: A Practical Guide**

May 30, 2026

**Reverse Engineering
Lab: Unpacking &
Neutralizing Android's Anti-
Emulator Toolkit**

May 30, 2026

Developed and maintained by [Vinay Vengala](#)